Module 7: Functions, Header Files & Distributed Computing

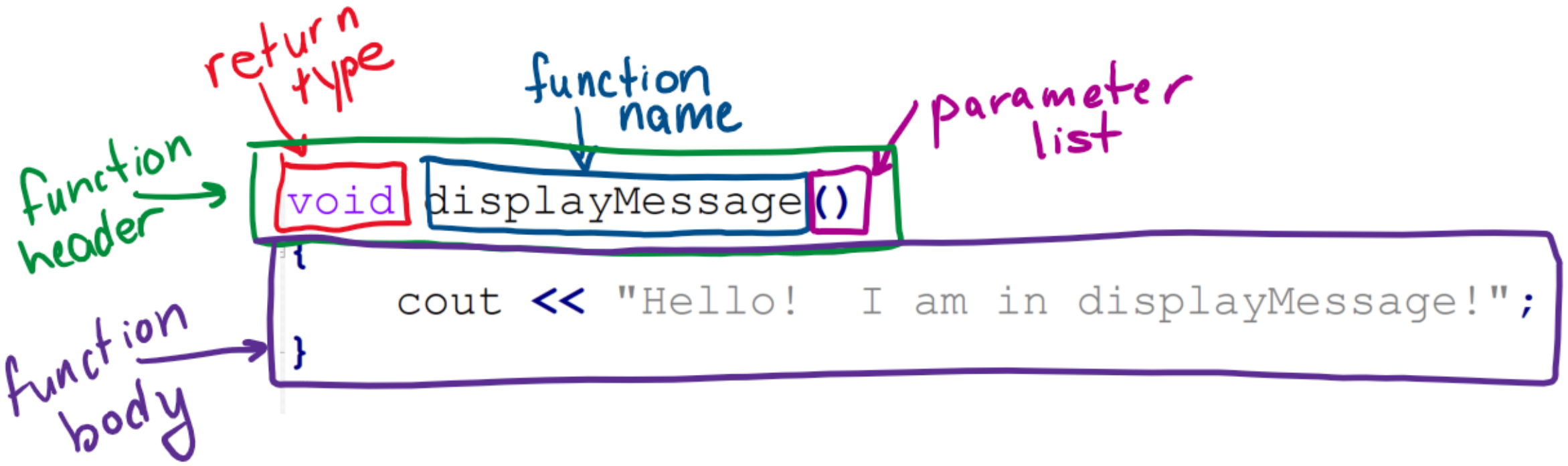
PURPLE GROUP

FUNCTIONS

Purpose of a Function

- A **function** is a named collection of statements that perform a task.
- *All functions other than the main function are called "User-Defined Functions" or "Programmer-Defined Functions."*
- *Functions* ***improve maintainability of programs.***
- *Functions* ***simplify the process of writing programs.***

Function Definition



Function Definition Example

Function definition: statements that make up a function

Basically, this IS the **function**.

```
void displayMessage()
{
      cout << "Hello! I am in displayMessage!";
}
```

Function Call Statement

Function call statement: statement causes a function to execute

This call statement is a line of code written in the main function or another programmer-defined function.

You just need the function name followed by () and then ;

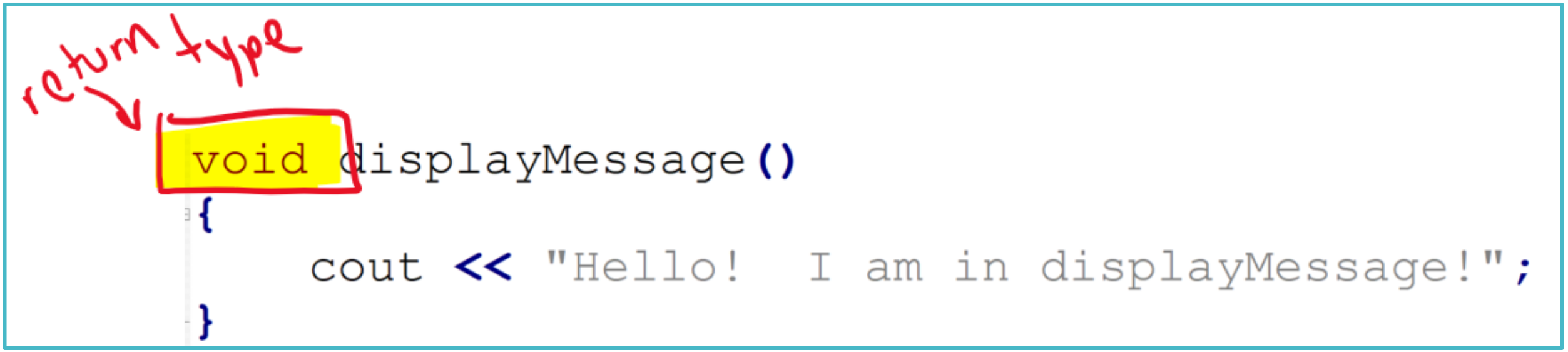
When called, the program executes the body of the called function.

After the function terminates, execution resumes in the calling function at point of call.

```
displayMessage();
```

Void Functions

If a function does not return a value, then its return type is **void**.



Function Examples

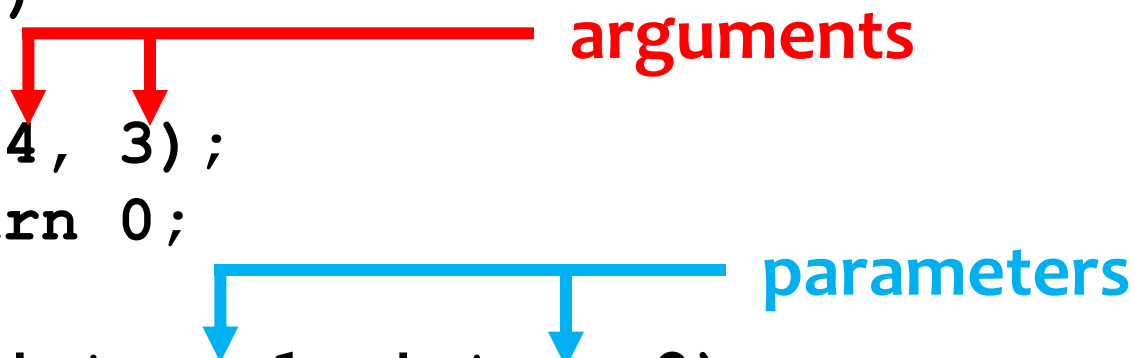
- **`functionExample_1.cpp`** – contains a user-defined function.
- **`functionExample_2.cpp`** – you should always put the main function first and all user-defined functions after that.
- **`functionExample_3.cpp`** – this is example 2 fixed with a function prototype.
- **`functionExample_4.cpp`** – a program with four user-defined functions and contains a function call statement in a user-defined function (instead of main)

Sending Data to Functions (pass by value)

Pass Data to a Function by Value

- Values passed to a function in the function call statement are called **arguments**.
- Variables in a function definition that hold the values passed to the function are called **parameters**.

```
int main()
{
      sum(4, 3);
      return 0;
}
void sum(int num1, int num2)
{
      cout << "The sum is " <<   (num1 + num2) << endl;
}
```



Pass by Value Examples

- **`passByValue_1.cpp`** – sending one integer to a function named displayValue
- **`passByValue_2.cpp`** – calling displayValue multiple times with a different argument each time
- **`passByValue_3.cpp`** – sending three integer arguments to a function named showSum
- **`passByValue_4.cpp`** – demonstrates changes to a parameter has no affect on original argument
- **`passByValue_5.cpp`** – menu-based program uses function to display menu and print results

Module 7 In-Class Practice 1
Passing Data by Value to a Function

DIRECTIONS

Create a C++ program called mod7a.cpp that contains a main function and one programmer-defined function as specified below.

In the `main` function:

Read in a string from the user and then call the **`countCharacters`** function, sending the string and the number 1. Then, call the **`countCharacters`** function again, sending the user's string and the number 2.

The `countCharacters` function:

This function should have a string and an integer parameter. If the integer is a **1**, count the number of **digits** in the string and **print** out the result.

If the integer is a 2, count the number of **characters that is not a space, letter, or digit**, and **print** out the result.

SAMPLE OUTPUT

```
Enter in a string: Th3b9#9ww@!9v_+34kI


The number of digits is 6

The number of characters other than a
space, letter, or digit is 5
```

Returning Data From Functions

Returning a Value from a Function

- A function can **return ONE value** back to the **function call statement.**
- You've already seen the **pow** function, which returns a value:

```
double x;
x = pow(2.0, 10.0);
```

return type

```
int sum(int num1, int num2)
{
    int result;
    result = num1 + num2;
    return result;
}
```

value being returned

Returning a Value from a Function
Returning an `int`

Functions can return the results of an arithmetic expression, such as `num1 + num2`

Example Function Call Statements:

```
int total, value1 = 4, value2 = 8;
total = sum(4, 8);
cout << "The total is " << total << endl;


cout << "The value is " << sum(value1, value2);
```

Example Function Definition:

```
int sum(int num1, int num2)
{
    return num1 + num2;
}
```

Returning a Value from a Function
Returning a `bool`

Functions can also return `true` or `false` (a Boolean)

Example Function Call Statement:

```
string passcode;
cout << "PASSCODE: ";
getline(cin, passcode);
if(allDigits(passcode))
      cout << "Valid passcode.";
```

Example Function Definition:

```
bool allDigits(string passcode)
{
      bool allDigits = true;
      for(int i=0; i<passcode.length(); i++){
            if(!isdigit(passcode.at(i)))
                  allDigits = false;
      }
      return allDigits;
}
```

Returning a Value from a Function using an Arithmetic Expressing

Functions can return the results of an arithmetic expression.

```
int sum(int num1, int num2)
{
    return num1 + num2;
}
```

result of the calculation is the value being returned

Returning a Value Examples

- **`returnData_1.cpp`**
- **`returnData_2.cpp`** – modified version of returnDat_1.cpp
- **`returnData_3.cpp`** – calculate area of circle using two functions
- **`returnData_4.cpp`** – converts cups to fluid ounces

Module 7 In-Class Practice 2 (part 1)
Returning Data From a Function

DIRECTIONS

- Write a program called **mod7b.cpp**.
- In the **`main()`** function, ask the user for their full name , favorite food, and favorite number.
- Then, call a function named **`luckyguess()`**, sending the number to this function. If this function returns **`true`**, print "[NAME], you will get a lifetime supply of [FOOD]!". Otherwise, print "[NAME], sorry, you get no [FOOD]."
- The **`luckyguess()`** function should have an **`int`** parameter (the user's number) and should return a **`bool`**. Generate and print a random number between 1 and 100. If the user is within 25 numbers of the generated number, then return **`true`**. Otherwise, return **`false`**.

SAMPLE OUTPUT 1

```
Name? April Crockett
Favorite Food? Chips & Salsa
Favorite Number? 7

Random number: 13

April Crockett, you will get a
lifetime supply of Chips &
Salsa!
```

SAMPLE OUTPUT 2

```
Name? Dwight Shrute
Favorite Food? Beet Juice
Favorite Number? 92

Random number: 43

Dwight Shrute, sorry, you get
no Beet Juice!
```

Pass by Reference Using Reference Variables

Purpose of Passing by Reference

- Using a **reference variable** as a parameter allows a function to work with the **original argument (variable)** from the function call, **not a copy** of the argument

- Allows the function to modify values stored in the calling environment (in the calling function)

- Provides a way for the function to 'return' more than one value

Reference Variables

- A **reference variable** is an **alias** (another name) for another variable.

- Defined with an ampersand (&). The example below shows a function prototype with one reference variable as a parameter. You can have an unlimited number of reference variables in a function.

```
void doubleNum(int&);
```

- Changes to a reference variable are made to the variable it refers to.

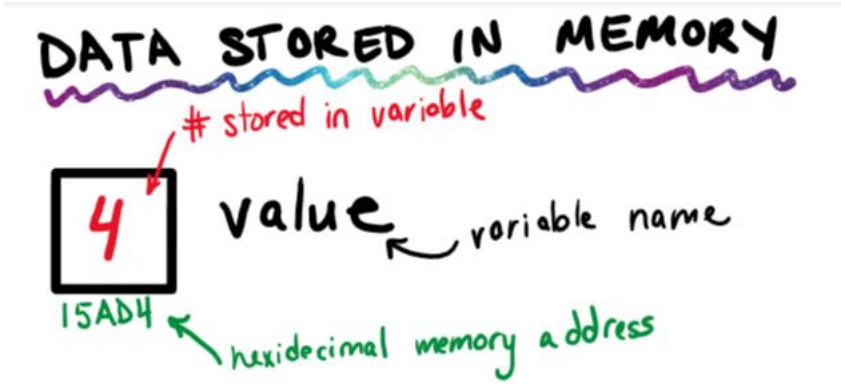
How to Use Reference Variables – slide 1 of 3

```
void doubleNum(int&);
int main()
{
      int value = 4;
      cout << "In main.\n";
      cout << "VALUE: " << value << endl;
      cout << "Now calling doubleNum";

      doubleNum(value);

      cout << "\nNow back in main.\n";
      cout << "VALUE: " << value << endl;
      return 0;
}
```

```
void doubleNum(int& num)
{
      num *= 2;
}
```




```
In main.
VALUE: 4
Now calling doubleNum
```

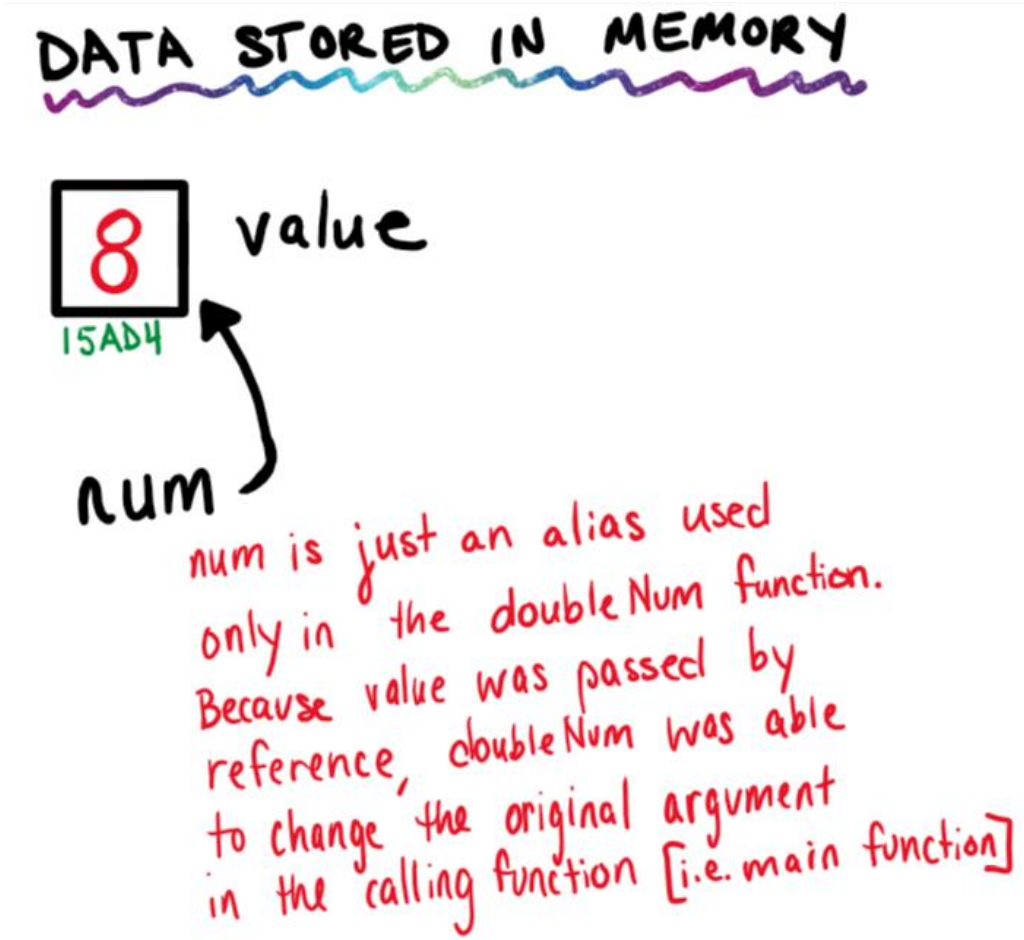
How to Use Reference Variables – slide 2 of 3

```
void doubleNum(int&);
int main()
{
      int value = 4;
      cout << "In main.\n";
      cout << "VALUE: " << value << endl;
      cout << "Now calling doubleNum";

      doubleNum(value);

      cout << "\nNow back in main.\n";
      cout << "VALUE: " << value << endl;
      return 0;
}
```

```
void doubleNum(int& num)
{
      num *= 2;
}
```



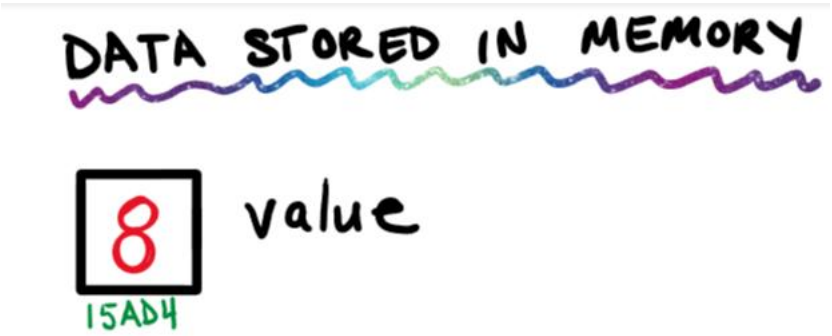
How to Use Reference Variables – slide 3 of 3

```
void doubleNum(int&);
int main()
{
      int value = 4;
      cout << "In main.\n";
      cout << "VALUE: " << value << endl;
      cout << "Now calling doubleNum";

      doubleNum(value);

      cout << "\nNow back in main.\n";
      cout << "VALUE: " << value << endl;
      return 0;
}
```

```
void doubleNum(int& num)
{
      num *= 2;
}
```




```
In main.
VALUE: 4
Now calling doubleNum
Now back in main.
VALUE: 8
```

Pass by Reference Examples

- **`referenceVariable_0.cpp`** – demonstrates purpose & syntax of using reference variables (best example of the three)
- **`referenceVariable_1.cpp`** – this is the program shown in the previous slides
- **`referenceVariable_2.cpp`** – a program containing two functions with reference variables.

Module 7 In-Class Practice **2 (part 2)**
Passing by Reference

DIRECTIONS

- Take your **mod7b.cpp** program and rename it to **mod7c.cpp**.
- Change the main function to call the **getUserData()** function before calling the **luckyguess()** function, sending the three variables to the function by reference (name, food, & number). Do not ask for user data in the main function.
- In the **getUserData()** function, you will ask the user for their full name , favorite food, and favorite number.

SAMPLE OUTPUT 1

```
Name? April Crockett
Favorite Food? Chips & Salsa
Favorite Number? 7

Random number: 13

April Crockett, you will get a lifetime supply
of Chips & Salsa!
```

SAMPLE OUTPUT 2

```
Name? Dwight Shrute
Favorite Food? Beet Juice
Favorite Number? 92

Random number: 43

Dwight Shrute, sorry, you get no Beet Juice!
```


Overloaded
Functions

Definition of Overloaded Functions

- Overloaded functions are two or more functions may have the **same name** if their **parameter lists** are **different**.
- Can be used to create functions that perform the same task but take different parameter types or different number of parameters
- Compiler will determine which version of function to call by argument and parameter lists

Overloaded Function
Function Prototypes & Call Statements

Using these overloaded functions,

```
void getDimensions(int);              // 1
void getDimensions(int, int);         // 2
void getDimensions(int, double);      // 3
void getDimensions(double, double); // 4
```

the compiler will use them as follows:

```
int length, width;
double base, height;
getDimensions(length);              // 1
getDimensions(length, width);       // 2
getDimensions(length, height);      // 3
getDimensions(height, base);        // 4
```

Overloaded Functions Example

- **`overloadedFunctions.cpp`**
  demonstrates purpose & syntax of using overloaded functions. This is a common case where you want to do the exact same operations, but with different data types.


Global &
Static Local
Variables
C++

Global Constants

- Variables are called global variables if they are defined outside of all functions (at the top of the source file).
- DO NOT USE global variables because they make programs difficult to debug.
- The only exception is global constants.

```
const double TAX_RATE = .0975;
```

- Example: **globalConstant.cpp**

Static Local Variables

- Local variables only exist while the function is executing. When the function terminates, the contents of local variables are lost.
- Static local variables retain their contents between function calls.
- Static local variables are defined and initialized only the first time the function is executed. (Zero is the default initialization value.)

Example: **`staticLocalVar.cpp`** 

Header Files

Header Files

In C++, there are two types of header files:

1. **Pre-existing header files:** files which are already available in C++ compiler we just need to import them.
   ```
   #include <filename>
   #include <iostream>
   ```

2. **User-defined header files:** these files are defined by the user and can be imported using "#include" too.
   ```
   #include "filename.h"
   #include "shapes.h"
   ```

What is a Header File?

The **#include** preprocessor directive directs the compiler that the header file needs to be processed before compilation.

What is In a Header File?

- Including pre-existing C++ header files
- Programmer-defined data types (coming in future modules)
- Global constant variables
- Function prototypes and/or function definitions
- Include guards (explained in a few slides)

Separating a Single Program in Multiple Files
1: Single Source File

```
1  #include <iostream>
2  using namespace std;
3  void doubleNum(int &);
4  void getNum(int &);
5  int main()
6  {
7      int value;
8
9      getNum(value);
10     doubleNum(value);
11     cout << "That value doubled is "
12     << value << endl;
13
14     return 0;
15 }
16 void getNum(int &userNum)
17 {
18    cout << "Enter a number: ";
19    cin >> userNum;
20 }
21 void doubleNum (int &refVar)
22 {
23    refVar *= 2;
24 }
```

First, here is a program all in a single source ".cpp" file.

In the next three slides you will be shown this program separated in to three separate files.

1. **Header file** called **referenceVars.h**
2. **Source file** with main function called **driver.cpp**
3. **Source file** with other function definitions called **functions.cpp**

Separating a Single Program in Multiple Files
1 – referenceVars.h (header file)

```
1  #include <iostream>
2  using namespace std;
3  void doubleNum(int &);
4  void getNum(int &);
5  int main()
6  {
7      int value;
8
9      getNum(value);
10     doubleNum(value);
11     cout << "That value doubled is "
12     << value << endl;
13
14     return 0;
15 }
16 void getNum(int &userNum)
17 {
18     cout << "Enter a number: ";
19     cin >> userNum;
20 }
21 void doubleNum (int &refVar)
22 {
23     refVar *= 2;
24 }
```

1. **Header file** called **referenceVars.h**

referenceVars.h

```
1  #ifndef REFERENCEVARS_H
2  #define REFERENCEVARS_H
3
4  #include <iostream>
5  using namespace std;
6
7  void doubleNum(int &);
8  void getNum(int &);
9
10 #endif
```

2. **Source file** with main function called **driver.cpp**

3. **Source file** with other functions called **functions.cpp**

These are called include guards. More about these in a few slides.

Separating a Single Program in Multiple Files
2 – driver.cpp (contains main function)

```
1  #include <iostream>
2  using namespace std;
3  void doubleNum(int &);
4  void getNum(int &);
5  int main()
6  {
7      int value;
8
9      getNum(value);
10     doubleNum(value);
11     cout << "That value doubled is "
12     << value << endl;
13
14     return 0;
15 }
16 void getNum(int &userNum)
17 {
18     cout << "Enter a number: ";
19     cin >> userNum;
20 }
21 void doubleNum (int &refVar)
22 {
23     refVar *= 2;
24 }
```

1. Header file called referenceVars.h

2. **Source file** with main function called **driver.cpp**

driver.cpp | referenceVars.h

LOOK!

```
1  #include "referenceVars.h"
2
3  int main()
4  {
5      int value;
6      getNum(value);
7      doubleNum(value);
8      cout << "That value doubled is "
9           << value << endl;
10     return 0;
11 }
```

3. **Source file** with other function definitions called **functions.cpp**

Separating a Single Program in Multiple Files
3 - functions.cpp

```
1  #include <iostream>
2  using namespace std;
3  void doubleNum(int &);
4  void getNum(int &);
5  int main()
6  {
7      int value;
8
9      getNum(value);
10     doubleNum(value);
11     cout << "That value doubled is "
12     << value << endl;
13
14     return 0;
15 }
16 void getNum(int &userNum)
17 {
18    cout << "Enter a number: ";
19    cin >> userNum;
20 }
21 void doubleNum (int &refVar)
22 {
23    refVar *= 2;
24 }
```

1. Header file called **referenceVars.h**
2. Source file with main function called **driver.cpp**
3. **Source file** with other function definitions called **functions.cpp**

functions.cpp | driver.cpp | referenceVars.h

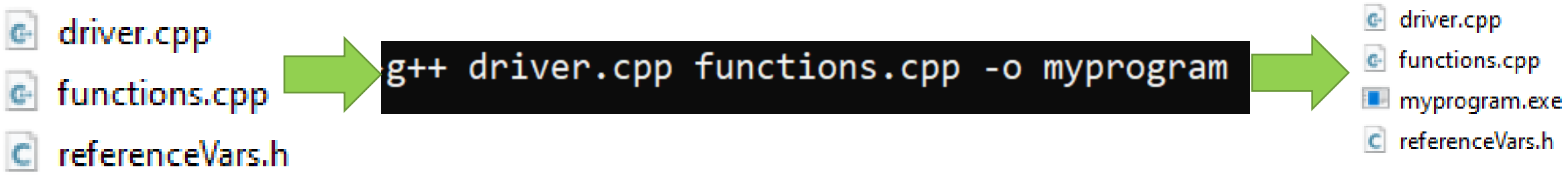
```
1  #include "referenceVars.h"
2
3  void getNum(int &userNum)
4  {
5     cout << "Enter a number: ";
6     cin >> userNum;
7  }
8
9  void doubleNum (int &refVar)
10 {
11    refVar *= 2;
12 }
```

LOOK!

How to Compile Multiple Files

When you have more then one source file to compile together, just list out **all the source files** after g++ **in any order**.

Notice you do not have to include the header file when compiling, because it is included within the code.



Include Guards in a Header File

- Source files (.cpp) in C++ can include any header file.
- A realistic program contains more then one source file (like example on previous slides).
- These source files could all include the same header file (like #include "referenceVars.h").
- Header files are only allowed to be included once and so having more then one .cpp file including the same header file leads to an error in the program. To eliminate this error, conditional preprocessor directives called "**Include Guards**" are used.

Syntax:

```
#ifndef HEADER_FILE_NAME
#define HEADER_FILE_NAME
      //the entire header file contents
#endif
```

Header File Program Example

- **Header Files Example** folder inside the "Module 7 Program Examples" zip file in ilearn.

Module 7 In-Class Practice 3 (part 1)
User-Defined Header Files

DIRECTIONS

- Make a copy of your **mod7c.cpp** program and paste it in a new folder named **mod7d**.
- Take your **mod7c.cpp** program and separate it out into three files as follows:
  - **luckyguess.h** (header file – contains #includes of pre-existing C++ header files, namespace, and function prototypes)
  - **driver.cpp** (source file – contains only the main function)
  - **functions.cpp** (source file – contains all the programmer-defined functions)
- Zip your **mod7d** folder and upload it.

SAMPLE OUTPUT 1

```
Name? April Crockett
Favorite Food? Chips & Salsa
Favorite Number? 7

Random number: 13

April Crockett, you will get a lifetime supply
of Chips & Salsa!
```

SAMPLE OUTPUT 2

```
Name? Dwight Shrute
Favorite Food? Beet Juice
Favorite Number? 92

Random number: 43

Dwight Shrute, sorry, you get no Beet Juice!
```

Makefiles

What is a Makefile?

- Manually compiling and linking a program with numerous source and header files can be time consuming and challenging.

- A Makefile is a special file used by the **`make`** utility to automate the process of compiling and linking programs.

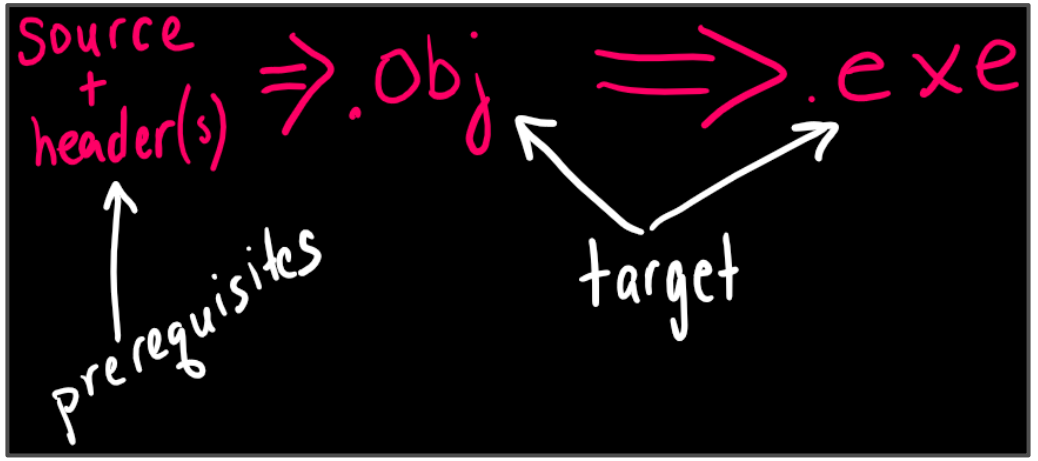
- The **`make`** utility uses a **Makefile** to recompile and link a program whenever changes are made to the source or header files.

Purpose and Benefits of using a Makefile

- **Saves time** by automating builds with a single command (make).
- Helps **manage multiple files** and their dependencies.
- Only recompiles the files that actually changed.
- Keeps your project **organized** and easier to maintain.
- Consistently compiles the project the same way every time.
- Makefiles are **portable**! You can use the same Makefile across different systems.

Makefile Rules and Commands

The **Makefile** contains **make rules** to specify dependencies between an object file and executable file and a set of prerequisite files (source & header files) that are needed to generate the object file.



Makefile Rule Syntax

Example Rule

```
target:  prerequisite_1  prerequisite_2…  prerequisite_n
     command_1
     command_2
```

Makefile Examples

Suppose your project includes the following files:

- **turtle.h** – contains your header file for your program
- **driver.cpp** – contains the main function
- **functions.cpp** – contains all your programmer-defined functions

To create a Makefile to help you compile & run your program, you would create a text file named "**Makefile**" with NO extension and place it in the same directory as your source and header files.

The following slides shows how to create a **Makefile for Windows** and then how to create a **Makefile for Mac**. They are very similar.

Makefile Example for Windows – slide 1 of 5

turtle.h

driver.cpp

functions.cpp

```
1  # ****************************WINDOWS VERSION of Makefile
2
3  all:          Turtleprog
4
5  Turtleprog: driver.cpp functions.cpp
6              g++ -Wall -o turtleprog driver.cpp functions.cpp
7
8  run:          turtleprog.exe
9              turtleprog
10
11 clean:        turtleprog.exe
12             del turtleprog.exe
```

Makefile Example for Windows – slide 2 of 5

turtle.h

driver.cpp

functions.cpp

```
1   # ************************************
2
3   all:            Turtleprog
4
5   Turtleprog: driver.cpp functions.cpp
6               g++ -Wall -o turtleprog driver.cpp functions.cpp
7
8   run:            turtleprog.exe
9               turtleprog
10
11  clean:          turtleprog.exe
12              del turtleprog.exe
```

`all` is the default rule and the first one in the Makefile. It's the entry point for the build process. When you type "**`make`**" with no arguments in the command prompt, this is the rule that executes.

This rule requires the **Turtleprog** rule to execute (next slide)

Makefile Example for Windows – slide 3 of 5

turtle.h

driver.cpp

functions.cpp

The Turtleprog rule executes only if driver.cpp and functions.cpp already exists.

This rule will compile the

table

```
1  # ************************WINDOWS VERSION of Makefile
2
3  all:          Turtleprog
4
5  Turtleprog: driver.cpp functions.cpp
6              g++ -Wall -o turtleprog driver.cpp functions.cpp
7
8  run:          turtleprog.exe
9              turtleprog
10
11 clean:        turtleprog.exe
12             del turtleprog.exe
```

Makefile Example for Windows – slide 4 of 5

turtle.h

driver.cpp

functions.cpp

```
1   # ***************************WINDOWS
2
3   all:            Turtleprog
4
5   Turtleprog: driver.cpp funct
6               g++ -Wall
7
8   run:            turtleprog.exe
9               turtleprog
10
11  clean:          turtleprog.exe
12              del turtleprog.exe
```

Type "`make run`" (without the double quotes) in the command prompt to RUN your program.

This rule will only execute if the turtleprog.exe file exists.

Makefile Example for Windows – slide 5 of 5

turtle.h

driver.cpp

functions.cpp

```
1   # ************************WINDOWS
2
3   all:            Turtleprog
4
5   Turtleprog: driver.cpp functi
6               g++ -Wall -o t
7
8   run:            turtl
9                   leprog
10
11  clean:          turtleprog.exe
12                  del turtleprog.exe
```

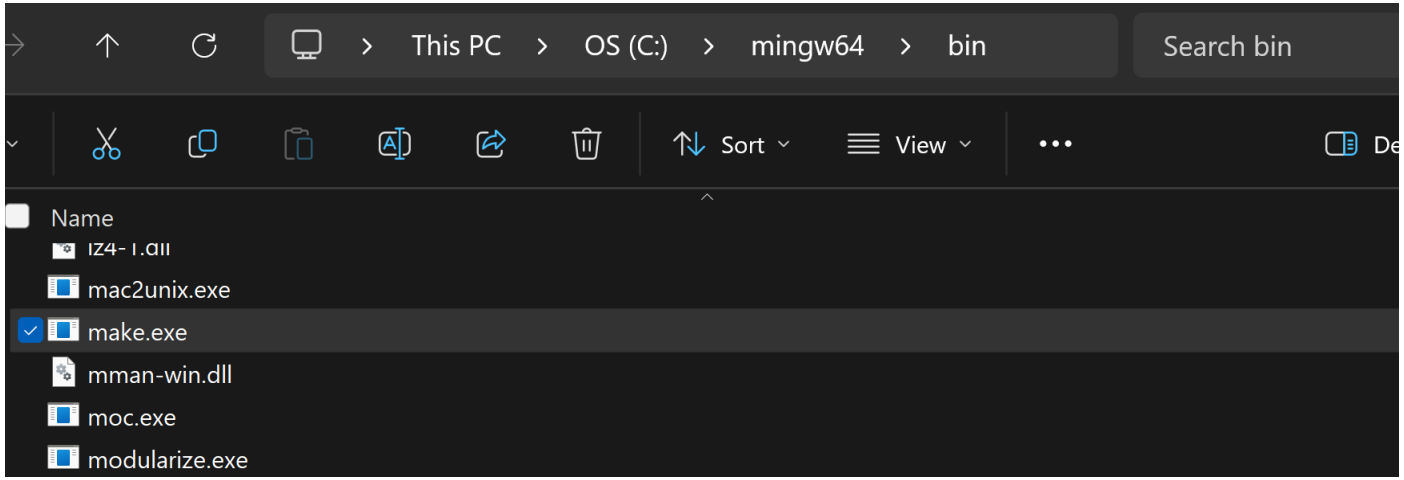
Type "`make clean`" (without the double quotes) in the command prompt to delete your executable file (turtleprog.exe).

This rule will only execute if the turtleprog.exe file exists.

Using Makefile on Windows (slide 1 of 4)

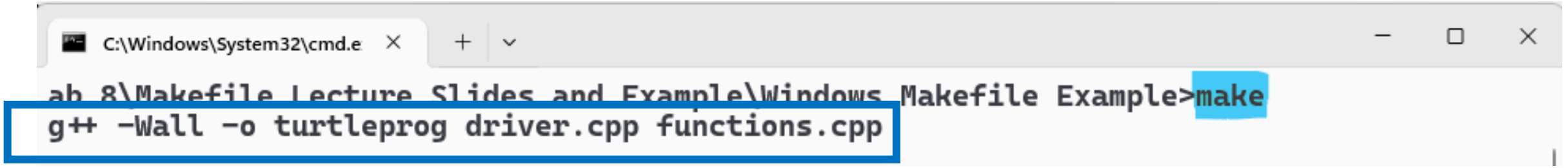
If you installed the MinGW compiler, it already came with make in the bin directory but it probably **needs to be renamed**.

1. Go to wherever you installed your mingw64 folder (mine is in C: drive)
2. Then click on the bin folder.
3. The files are in alphabetical order. Scroll way down to the m's.
4. Rename **`mingw32-make`** to just **`make`**



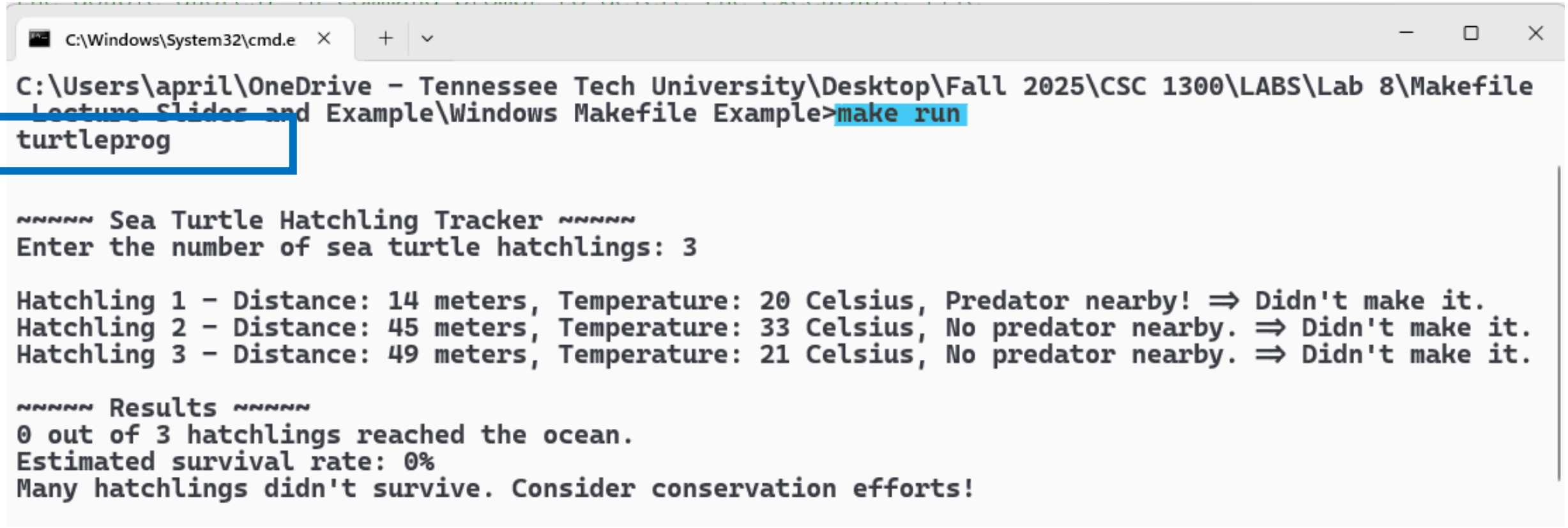
Using Makefile on Windows (slide 2 of 4)

Now go to the directory where your program (and Makefile) are located in your command prompt or terminal and type **`make`**. This creates your executable file.



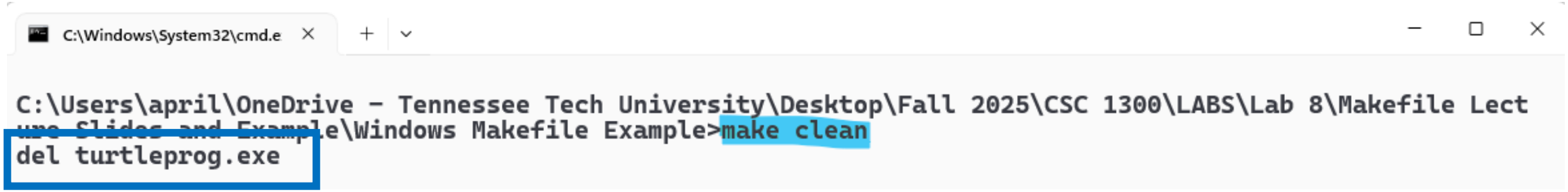
Using Makefile on Windows (slide 3 of 4)

Now **run** your program by typing `make run` in your command prompt.



Using Makefile on Windows (slide 4 of 4)

If you want to delete your executable file, type **`make clean`** in your command prompt.



Makefile Example for Mac – slide 1 of 5

turtle.h

driver.cpp

functions.cpp

```
1  # ****************************MAC VERSION of Makefile
2  all:          Turtleprog
3
4  Turtleprog: driver.cpp functions.cpp
5              g++ -Wall -o turtleprog driver.cpp functions.cpp
6
7  run:          turtleprog
8              ./turtleprog
9
10 clean:        turtleprog
11             rm -f turtleprog
```

Makefile Example for Mac – slide 2 of 5

turtle.h

driver.cpp

functions.cpp

```
1   #  ********************
2   all:
3
4   Turtleprog: driver.cpp functions.cpp
5               g++ -Wall -o turtleprog driver.cpp functions.cpp
6
7   run:        turtleprog
8               ./turtleprog
9
10  clean:      turtleprog
11              rm -f turtleprog
```

all is the default rule and the first one in the Makefile. It's the entry point for the build process. When you type "**make**" with no arguments in the terminal, this is the rule that executes.

This rule requires the **Turtleprog** rule to execute (next slide)

Makefile Example for Mac – slide 3 of 5

turtle.h

driver.cpp

functions.cpp

```
1   # ****************************
2   all:            Turtlepro
3
4   Turtleprog: driver.cpp functions.cpp
5               g++ -Wall -o turtleprog driver.cpp functions.cpp
6
7   run:            turtleprog
8                   ./turtleprog
9
10  clean:          turtleprog
11                  rm -f turtleprog
```

The Turtleprog rule executes only if driver.cpp and functions.cpp already exists.

This rule will compile the source files into an executable file named `turtleprog`

Makefile Example for Mac – slide 4 of 5

turtle.h

driver.cpp

functions.cpp

```
1   # ************************************MAC V
2   all:            Turtleprog
3
4   Turtleprog: driver.cpp
5               g++                          leprog
6
7   run:            turtleprog
8                   ./turtleprog
9
10  clean:          turtleprog
11                  rm -f turtleprog
```

Type "`make run`" (without the double quotes) in the terminal to RUN your program.

This rule will only execute if the `turtleprog` executable file exists.

Makefile Example for Mac – slide 5 of 5

turtle.h
driver.cpp
functions.cpp

```
1   # ***************************MAC V
2   all:            Turtleprog
3
4   Turtleprog: driver.cpp funct
5               g++ -Wall -
6
7   run:            tur
8                   urtleprog
9
10  clean:          turtleprog
11                  rm -f turtleprog
```

Type "`make clean`" (without the double quotes) in the terminal to delete your executable file (`turtleprog`).

This rule will only execute if the `turtleprog` executable file exists.

Using Makefile on Mac

Xcode already comes with make. To verify in your terminal, type

```
make -version
```

Then, you can duplicate the directions for Windows by typing **`make`** to create your executable and then run the program as usual.

Using Makefile on Linux

Ensure build-essential is installed. To verify in your terminal type

```
sudo apt update
```

```
sudo apt install build-essential
```

To verify make is installed in your terminal, type

```
make -version
```

Then, you can duplicate the directions for Windows by typing `make` to create your executable and then run the program as usual.

Distributed Data

Distributed Data

- In real world programs, data is rarely all located on a programmer's local machine. Typically, data is remote, and you must transfer or retrieve the data to access or modify it on your local machine.
- **Distributed data** is data that is spread across multiple systems, computing notes, or data storage locations.

Reasons for Accessing Remote Data Services

List of circumstances where a computing problem requires interaction with a remote service:

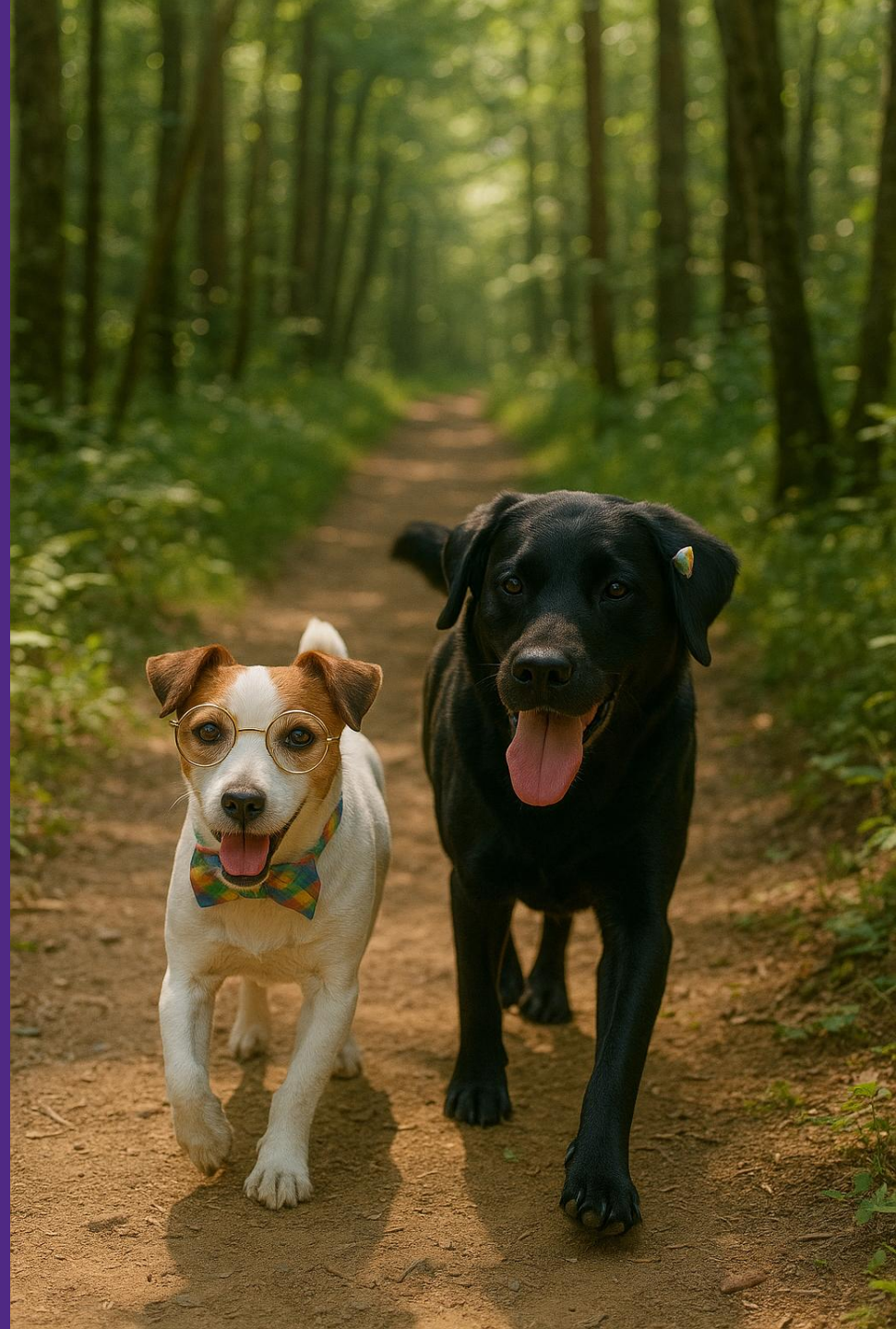
1. **Data Source Requirements** – if the data required to solve a computing problem resides on an external or remote source or needs to integrate with an external (or third-party system/service) API (e.g., payment gateways, social media APIs, mapping services)
2. **Real-Time or Up-to-Date Information** – if the solution requires real-time or up-to-date data that changes frequently. Some examples include stock prices, social media posts, weather information, etc.)
3. **Functionality Beyond Local Capabilities** – if the functionality needed to solve the problem exceeds the local environment capabilities in terms of performance, bandwidth, or storage (e.g., processing large datasets or performing complex calculations). The data could be stored on supercomputer remote servers.
4. **Security and Authentication** – if the solution involves accessing secure or authenticated information (e.g., user accounts, private databases), this often requires interaction with remote services that manage security.

Reasons for Accessing Remote Data Services, cont.

Overall, the need for remote service interaction arises when the problem solution involves:

- accessing data or capabilities that are beyond the immediate boundaries of the application or system, or
- when real-time, authenticated, or scalable solutions are required.

JSON



What is JSON?

- **JSON** is pronounced “Jason” and stands for **J**ava**S**cript **O**bject **N**otation.
- JSON is a **text-based data format** that is commonly used in many applications for storing and **transporting data** between servers and web applications and is important in software using distributed data.

Douglas Crockford is an American computer programmer who is involved in the development of the JavaScript Language and **specified the data format JSON.**

Douglas Crockford

Image By By Robert Claypool - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=24517464

JSON Example & Parsing

JSON text is very readable by both humans and computers.

This example is a JSON string:

```
'{"name":"John", "age":30, "car":null}'
```

This string defines an **object** with 3 **properties**: name, age, & car.

Each property has a value. Each property is made up of a **key-value pair**. For example, the key "name" has value "John".

If you parse (go through each part) the JSON string with a program, you can access the data as an object. **Code for reading and generating JSON exists in many programming languages.**

JSON Syntax Rules

- Data is in **key/value** pairs.
  - **Keys** must be strings, written with double quotes.
  - **Values** must be a string, a number, an object, an array, a Boolean, or null.
- Data is separated by **commas**.
- **Curly braces {}** hold objects.
- **Square brackets []** hold arrays.


```
{"employees":[
  { "firstName":"John", "lastName":"Doe" },
  { "firstName":"Anna", "lastName":"Smith" },
  { "firstName":"Peter", "lastName":"Jones" }
]}
```

Getting JSON data with C++ Code

- There are many JSON libraries available, but the most commonly used is "JSON for Modern C++ version 3.11.3" (as of July 2024) and is available on GitHub made by Dr. Lohmann at https://github.com/nlohmann/json
- Documentation for this library is at https://github.com/nlohmann/json/blob/develop/README.md

Dr. Niels Lohmann is from Berlin, Germany who created the commonly used JSON library for C++.

Niels Lohmann

Image from Niels Lohmann's GitHub Repository:
https://avatars.githubusercontent.com/u/159488?v=4

How To Set Up Your Files & Code to Use JSON for Modern C++

- To use the JSON for Modern C++ library, download the code at https://github.com/nlohmann/json and place it in a json folder in the same directory as your source file(s).
- In your code that uses this library, you will need to have this included:
  `#include <nlohmann/json.hpp>`
- When you compile, you will need to add on the library with a –I flag like this:
  `g++ sourcefile.cpp -I "C:/PATH-TO-FILES/json/include"`
  but change "PATH-TO-FILES" to the actual path where your json folder is located.

How To Set Up Your Files & Code to Use JSON for Modern C++, cont.

- To use the JSON for Modern C++ library, download the code at https://github.com/nlohmann/json and place it in a json folder in the same directory as your source file(s).
- In your code that uses this library, you will need to have this included:
```
#include <nlohmann/json.hpp>
```
- In your code after your #includes, you will also need the line below, which is a *namespace-alias-definition* to make *json* a synonym for the *nlohmann::json* namespace.
```
using json = nlohmann::json;
```
- When you compile, you will need to add on the library with a –I flag like this:
```
g++ sourcefile.cpp –I "C:/PATH-TO-FILES/json/include"
```
but change "PATH-TO-FILES" to the actual path where your json folder is located.

Getting Data from an HTML Web Page with cURL

- To retrieve data from a website and store it in your C++ program, you can use the open source common **c**lient **URL** (**cURL**), which is a command-line tool that is available for Linux, Mac, and Windows.
- cURL website: https://curl.se/
- This tool allows developers to send and receive data between a device and a server.
- cURL works by specifying a server's location (**URL**) and the data to be sent through a terminal.
- cURL can use different protocols such as HTTP, HTTPS, and FTP, to send and receive data in various formats, including files, JSON, and form data.
- cURL flags can also be used to filter the server's response, including the status code, headers, and response body.

End of Module 7